CAPACITACIÓN · LARAVEL

Buenas prácticas y arquitectura de software con Laravel

Ejemplos sencillos para diseñar aplicaciones claras, mantenibles y extensibles.

Luis Eduardo Álvarez Henao

Senior Full Stack Developer

Más de 10 años de experiencia en desarrollo de software, especializado en arquitectura de aplicaciones, soluciones empresariales e integración de sistemas.

Tecnologías: Laravel, PHP, Node.js, TypeScript, Docker y AWS.

LinkedIn: linkedin.com/in/luis-eah

Rol y permiso: Ana solo necesita consultar usuarios

Rol: lector

Agrupa permisos de consulta. En este ejemplo tiene users.index, que permite ver la lista de usuarios.

Permiso: users.index

Autoriza una acción concreta: consultar la lista. users.create y users.update son otros permisos; el rol lector no los tiene.

Ejemplo: Ana es lectora

Ana recibe el rol lector. Puede consultar usuarios, pero no crearlos ni editarlos. Para permitir una acción nueva, se concede su permiso.

Rol = conjunto de permisos. Permiso = una acción. $ana->can('users.index') devuelve true; $ana->can('users.create') devuelve false en este ejemplo.

Añadir un lector sin cambiar cada ruta

MALA PRÁCTICA: CAMBIAR CONDICIONES

```
// Antes: solo admin consulta usuarios
if ($user->hasRole('admin')) {
    // mostrar la lista de usuarios
}

// Ahora también debe entrar el rol lector
if ($user->hasRole('admin')
    || $user->hasRole('lector')) {
    // mostrar la lista de usuarios
}
// cada rol nuevo obliga a revisar condiciones
```

BUENA PRÁCTICA: ASIGNAR EL PERMISO

```
// Se pregunta por la acción permitida
if ($user->can('users.index')) {
    // mostrar la lista de usuarios
}

// El permiso ya existe: se concede al rol
Role::findOrCreate('lector')
    ->givePermissionTo('users.index');

$ana->assignRole('lector');
// Ana consulta, sin tocar la ruta
```

EJEMPLO Y RESULTADO

Admin ya tiene users.index. Al dárselo al rol lector, Ana también puede ver la lista. La ruta sigue usando el mismo can().

TEN EN CUENTA

La comprobación expresa qué acción está permitida. El nombre del rol puede cambiar sin reescribir la autorización de cada ruta.

ACL · POLICY

Editar a Luis: policy y FormRequest

01 · POLICY: PERMISO Y ESTADO

```
// UserPolicy: $actor intenta editar a $target
public function update(User $actor, User $target)
{
    if (! $actor->can('users.update')) {
        return Response::deny('Sin permiso');
    }

    return $target->is_active
        ? Response::allow()
        : Response::deny('El usuario está inactivo');
}
// Response: Illuminate\Auth\Access\Response
```

02 · FORMREQUEST: USA LA POLICY

```
class UpdateUserRequest extends FormRequest
{
    public function authorize(): Response
    {
        return Gate::inspect('update', $this->route('user'));
    }

    public function rules(): array
    {
        return ['name' => ['required', 'string', 'max:255']];
    }
}
```

EJEMPLO Y RESULTADO

Ana, lectora sin users.update: 403 "Sin permiso". Carlos con permiso y Luis inactivo: 403 "El usuario está inactivo". Carlos con permiso y Luis activo: 200.

TEN EN CUENTA

Gate::inspect devuelve la respuesta de la policy: el 403 trae su mensaje. Un nombre vacío responde 422 y no se guarda nada.

Crear usuarios: el botón muestra; el servidor autoriza

MALA PRÁCTICA: SOLO EL BOTÓN

```
// React solo oculta el botón
{can('users.create') && <CreateUserButton />}

// El servidor guarda sin comprobar el permiso
public function store(Request $request)
{
    $data = $request->validate([
        'name' => ['required', 'string', 'max:255'],
    ]);

    User::create($data);
}
// llamar a la API directamente evita el botón
```

BUENA PRÁCTICA: AUTORIZAR EN EL SERVIDOR

```
class StoreUserRequest extends FormRequest
{
    public function authorize(): bool
    {
        return $this->user()->can('users.create');
    }

    public function rules(): array
    {
        return ['name' => ['required', 'string', 'max:255']];
    }
}
// el controlador recibe StoreUserRequest
```

EJEMPLO Y RESULTADO

Sin permiso, POST /api/v1/users responde 403, esté el botón oculto o no. Con el permiso y datos válidos, guarda.

TEN EN CUENTA

React oculta el botón para orientar al usuario; el servidor decide. En el frontend, can() solo sirve para mostrar u ocultar.

Probar el control de acceso: tres casos

MALA PRÁCTICA: PROBAR SOLO CON ADMIN

```
public function test_updates_a_user(): void
{
    $admin = User::factory()->create();
    $admin->assignRole('admin');

    Sanctum::actingAs($admin);
    $this->putJson('/api/v1/users/15', $datos)
        ->assertOk();
}
// el admin pasa todo: si users.update se
// escribe mal, la prueba no lo detecta
```

BUENA PRÁCTICA: SIN SESIÓN, SIN PERMISO, CON PERMISO

```
// 1. sin sesión
$this->putJson('/api/v1/users/15', $datos)
    ->assertUnauthorized();

// 2. con sesión, sin permiso (Ana, lectora)
Sanctum::actingAs($ana);
$this->putJson(...)->assertForbidden();

// 3. con el permiso exacto (Carlos)
$carlos->givePermissionTo('users.update');
Sanctum::actingAs($carlos);
$this->putJson(...)->assertOk();
```

EJEMPLO Y RESULTADO

401 sin sesión, 403 sin el permiso y 200 con el permiso exacto. Cada prueba concede únicamente lo que necesita.

TEN EN CUENTA

Agrega también el caso de dominio (por ejemplo, no poder borrarse a uno mismo, incluso siendo admin).

API / VISTAS WEB

La web entrega la página; la API entrega los datos

ANTIPATRÓN

```
// Mezclar Inertia y JSON en la misma acción
if ($request->wantsJson()) {
    return response()->json($users);
}

return Inertia::render('users/index', [
    'users' => $users,
]);
```

PATRÓN DEL PROYECTO

```
// Web: routes/web.php (web + auth)
Gate::authorize('update', $user);
return Inertia::render('users/form', [
    'userId' => $user->id,
]);

// API: routes/api.php (auth:sanctum, /api/v1)
$users = User::query()
    ->applyFilters($request)
    ->paginated(15)
    ->through(fn ($u) => UserResource::make($u)->resolve());
return Response::api($users, __('Successful'));
```

EJEMPLO Y RESULTADO

Cuando hay páginas y API: controladores en Web y Api/V1, y un único contrato de datos para React, móvil u otros sistemas.

TEN EN CUENTA

En una página sencilla sin API, pasar los datos como props de Inertia puede ser suficiente.

El recurso va en la URL; la acción, en el verbo HTTP

Acción	Verbo y ruta	Método
Listar usuarios	GET /api/v1/users	index
Ver un usuario	GET /api/v1/users/{user}	show
Crear un usuario	POST /api/v1/users	store
Actualizar un usuario	PUT /api/v1/users/{user}	update
Eliminar un usuario	DELETE /api/v1/users/{user}	destroy

Sustantivos en plural y kebab-case (/request-types). Anida poco. GET nunca cambia datos. Versionar (/v1) permite que v2 conviva con v1 para clientes externos.

CONTROLADOR

Crear un usuario: recibir, guardar y responder

MALA PRÁCTICA: TODO EN UNA ACCIÓN

```
public function store(Request $request)
{
    // Verificar permisos aquí
    // Validar aquí
    // Normalizar campos aquí
    // Guardar y preparar el JSON aquí
    // Copiar todo en otra ruta después
}
// todo cambia en el mismo método
```

BUENA PRÁCTICA: ORQUESTAR

```
public function store(StoreUserRequest $request)
{
    // el FormRequest ya autorizó y validó
    $user = User::create($request->validated());

    return Response::api(
        new UserResource($user),
        __('Usuario creado'), status: 201
    );
}
// caso sencillo: no necesita una acción extra
```

EJEMPLO Y RESULTADO

POST /api/v1/users crea al usuario y devuelve 201. El FormRequest valida, el Resource elige los campos y Response::api unifica el formato.

TEN EN CUENTA

Si crear también toca otras tablas o coordina una transacción, extrae ese trabajo a una acción. No agregues capas a un CRUD sencillo.

API · RESPUESTA ESTÁNDAR

Tres llamadas, tres respuestas con el mismo formato

CÓDIGO · ELIGE EL CASO

```
// 1. Crear: datos y mensaje
return Response::api(
    new UserResource($user),
    __('Usuario creado'), status: 201
);

// 2. Confirmar: solo mensaje
return Response::success(__('Usuario actualizado'));

// 3. Rechazar: mensaje y código HTTP
return Response::error(__('No permitido'), 403);
```

RESPUESTA · MISMA NUMERACIÓN

```
// 1. HTTP 201
{ "status": "success", "message": "Usuario creado",
  "data": { "id": 15, "name": "Ana" } }

// 2. HTTP 200
{ "status": "success",
  "message": "Usuario actualizado" }

// 3. HTTP 403
{ "status": "error",
  "message": "No permitido" }
```

EJEMPLO Y RESULTADO

__('Usuario creado') busca la traducción del idioma activo; en inglés devuelve User created. Resource define data y Response::api unifica el formato.

TEN EN CUENTA

Response::api, success y error son macros del paquete module/api-response de este proyecto; no son métodos nativos de Laravel.

API · SALIDA JSON

Devuelve los campos que necesita la pantalla

MALA PRÁCTICA: RESPUESTAS DISTINTAS

```
// La respuesta depende de la forma del modelo
return response()->json($user);

// O cada controlador arma una forma distinta
return [
    'user_id' => $user->id,
    'nombre' => $user->name,
];
// otra ruta devuelve id y name
```

BUENA PRÁCTICA: CAMPOS EXPLÍCITOS

```
// UserResource: campos elegidos explícitamente
public function toArray(Request $request): array
{
    return [
        'id' => $this->id,
        'name' => $this->name,
        'email' => $this->email,
    ];
}

// el controlador usa esa misma representación
return Response::api(new UserResource($user));
```

EJEMPLO Y RESULTADO

Usuario 15: { "id": 15, "name": "Ana", "email": "ana@example.com" }. La pantalla recibe nombres de campos previsibles.

TEN EN CUENTA

response()->json($user) puede exponer columnas que no debían salir (password, remember_token) y las que se agreguen mañana.

nullable permite null; sometimes valida solo si viene

01 · NULLABLE: EL CAMPO PUEDE SER NULL

```
// Se puede borrar el teléfono
'phone' => ['nullable', 'string', 'max:20'],

// { "phone": "300 123 4567" }  => válido
// { "phone": null }            => válido (se borra)
// { }                          => válido (no se toca)

// sin nullable, { "phone": null } falla:
// "debe ser una cadena de texto"
```

02 · SOMETIMES: REGLAS SOLO SI EL CAMPO VIENE

```
// PATCH: solo viaja lo que cambia
'name' => ['sometimes', 'required', 'string'],

// { "name": "Luis" }  => válido
// { }                 => válido (sometimes salta
//                        las reglas si no viene)
// { "name": null }    => falla (required)

// sin sometimes, { } falla por required
```

EJEMPLO Y RESULTADO

PATCH /api/v1/users/15 con { } es válido y no cambia nada. Con { "name": "" } responde 422: Laravel convierte "" en null y required lo rechaza.

TEN EN CUENTA

nullable responde a ¿puede venir null? sometimes responde a ¿se valida si no viene? Se combinan: ['sometimes', 'nullable', 'string'].

Valida lo que el cliente puede cargar, ordenar y paginar

ANTIPATRÓN

```
// El modelo carga siempre las relaciones
class User extends Model
{
    protected $with = ['roles', 'tokens'];
}

// cada endpoint las recibe, las use o no
User::query()->get();   // consultas extra en todos

// o relaciones arbitrarias desde la petición
User::query()->with($request->input('with'))->get();
// un nombre inventado puede producir un 500
```

PATRÓN DEL PROYECTO

```
// UserIndexRequest
'with' => ['nullable', 'array'],
'with.*' => ['distinct', Rule::enum(UserRelation::class)],
'sort.*' => [Rule::enum(SortDirection::class)],

// Builder: carga solo lo que el cliente pide
User::query()
    ->applyFilters($request)
    ->paginated(15);   // per_page acotado a 100

// GET /api/v1/users               => sin relaciones
// GET /api/v1/users?with[]=roles  => con roles
```

EJEMPLO Y RESULTADO

Cada endpoint decide: la lista de usuarios pide roles; otra pantalla que no los muestra no paga esa consulta. Una relación fuera del enum responde 422.

TEN EN CUENTA

No dejes with fijos en el modelo ($with) ni en el controlador: cargan datos que quizá ese endpoint no necesita. Sanctum usa sesión y cookie para la SPA propia, y tokens para clientes externos.

DATOS · CONVENCIONES

Deja que Laravel deduzca los nombres

Elemento	Convención	Ejemplo
Modelo / tabla	Singular StudlyCase / plural snake_case	Student / students
Clave primaria / foránea	id / {modelo_singular}_id	id / request_type_id
Tabla pivote	Singular y orden alfabético	agreement_institution
Booleano	Prefijo is_ o has_	is_active / is_local
Momento / fecha	Sufijo _at / _on	submitted_at / starts_on
Marcas de tiempo / borrado	created_at, updated_at / deleted_at	timestamps() / softDeletes()
Estados / datos libres	string + enum de PHP / json o jsonb	status / metadata

Si te apartas de una convención, documenta el motivo. AdministrativeStaff usa administrative_staff porque staff no se pluraliza.

Un estado conocido en PHP y una columna simple en la base

ANTIPATRÓN

```
// Los valores quedan definidos en la base
$table->enum('status', ['draft', 'published']);

// Cada valor nuevo exige alterar la columna
// y los textos se repiten en el código
if ($article->status === 'publshed') {
    // error de escritura: nunca coincide
}
```

PATRÓN DEL PROYECTO

```
// Base de datos
$table->string('status', 20)->default('draft');

enum ArticleStatus: string
{
    case Draft = 'draft';
    case Published = 'published';
}
// Article::casts()
return ['status' => ArticleStatus::class];
// FormRequest::rules()
'status' => [Rule::enum(ArticleStatus::class)],
```

EJEMPLO Y RESULTADO

draft se convierte en ArticleStatus::Draft; un valor como publshed se rechaza al validar. El Resource devuelve $this->status->value.

TEN EN CUENTA

Fechas: published_at incluye hora; starts_on solo fecha. Booleanos: is_active. Si el usuario crea valores nuevos, usa una tabla de catálogo.

BASE DE DATOS · INTEGRIDAD

La clave foránea también define qué pasa al borrar

ANTIPATRÓN

```
$table->unsignedBigInteger('author_id');

// nada impide guardar un autor inexistente
// borrar al autor puede dejar huérfanos
```

PATRÓN DEL PROYECTO

```
$table->foreignId('article_id')
    ->constrained()->cascadeOnDelete();

$table->foreignId('author_id')
    ->constrained('users')->restrictOnDelete();

$table->foreignId('reviewer_id')->nullable()
    ->constrained('users')->nullOnDelete();

// cascade: el hijo depende del padre
// restrict: protege; null: vínculo opcional
```

EJEMPLO Y RESULTADO

Un comentario depende de su artículo (cascade). Un usuario con artículos no se borra (restrict). Sin revisor, el vínculo queda en null.

TEN EN CUENTA

Indica la tabla cuando el nombre de la columna no permite deducirla. Si una columna apunta a varias tablas, valídala en el código.

Una migración nueva para cada cambio compartido

ANTIPATRÓN

```
// Editar una migración que ya corrió
// el nuevo campo no llegará a otros entornos
$table->string('status');

// Usar modelos que cambiarán con el tiempo
Article::all()->each(fn ($a) =>
    $a->update(['summary' => '...'])
);
```

PATRÓN DEL PROYECTO

```
// Migración nueva, repetible
if (Schema::hasColumn('articles', 'status')) {
    return;
}

Schema::table('articles', function ($table) {
    $table->string('status', 20)
        ->default('draft')->index();
});

// transformaciones: DB::table(), sin Eloquent
// down() cuando sea posible revertir
```

EJEMPLO Y RESULTADO

Una migración, un propósito. Artisan genera el nombre y la fecha. Con datos existentes, la columna nueva necesita default o nullable.

TEN EN CUENTA

Nunca edites una migración ya ejecutada en un entorno compartido. Los seeders también deben poder repetirse.

Ponle nombre al filtro que repites: publicados

MALA PRÁCTICA: FILTROS COPIADOS

```
// En el inicio
Article::where('status', 'published')
    ->where('published_at', '<=', now())->get();

// En el buscador se olvida la fecha
Article::where('status', 'published')->get();
// aparece un artículo programado para mañana
```

BUENA PRÁCTICA: FILTRO CON NOMBRE

```
// En ArticleBuilder
public function published(): static
{
    return $this->where('status', 'published')
        ->where('published_at', '<=', now());
}

// Article usa ese builder personalizado
Article::query()->published()->paginate(15);
// inicio y buscador comparten el filtro
```

EJEMPLO Y RESULTADO

Un artículo con fecha de mañana no aparece hoy. Cambiar la regla de publicación requiere modificar una sola consulta con nombre.

TEN EN CUENTA

Usa un builder o scope cuando el filtro se repite o expresa una regla del negocio. Un where() aislado puede quedarse donde se usa.

Crear un artículo y sus etiquetas como una sola operación

MALA PRÁCTICA: GUARDADOS SUELTOS

```
// Dentro del controlador
$article = Article::create($request->all());
$article->tags()->sync($request->tags);

// Si falla la segunda escritura,
// el artículo ya quedó guardado
// Otra entrada repite este procedimiento
```

BUENA PRÁCTICA: TRANSACCIÓN EN LA ACCIÓN

```
// CreateArticleData: DTO con title, body y tagIds
public function handle(CreateArticleData $data): Article
{
    return DB::transaction(function () use ($data) {
        $article = Article::create([
            'title' => $data->title,
            'body' => $data->body,
        ]);
        $article->tags()->sync($data->tagIds);

        return $article;
    });
}
```

EJEMPLO Y RESULTADO

Si falla la asociación de etiquetas, se revierte la creación del artículo. El DTO entrega a la acción los campos que necesita.

TEN EN CUENTA

Úsalo al coordinar varias escrituras o entradas. El controlador crea el DTO desde validated() y llama a handle().

PATRÓN · STRATEGY

Una tienda calcula el envío normal o urgente

ANTIPATRÓN: UN IF QUE CRECE

```
function precioEnvio(string $tipo): int
{
    if ($tipo === 'normal') {
        return 5000;
    } elseif ($tipo === 'urgente') {
        return 12000;
    }
    // cada tipo nuevo obliga a editar esta función
}
```

PATRÓN: UNA CLASE POR VARIANTE

```
interface TarifaEnvio
{
    public function precio(): int;
}

class EnvioNormal implements TarifaEnvio
{
    public function precio(): int { return 5000; }
}

class EnvioUrgente implements TarifaEnvio
{
    public function precio(): int { return 12000; }
}
```

EJEMPLO Y RESULTADO

Cada forma de envío es una clase con su precio. Para un envío nuevo se agrega una clase; la función que calcula no cambia.

TEN EN CUENTA

Úsalo cuando hay varias formas de hacer lo mismo y se agregan más. Con dos variantes fijas, un if puede ser más claro.

PATRÓN · STRATEGY + ENUM

El enum elige la estrategia; cada clase calcula

01 · EL ENUM ELIGE EL ENVÍO

```
enum TipoEnvio: string
{
    case Normal = 'normal';
    case Urgente = 'urgente';

    public function estrategia(): TarifaEnvio
    {
        return match ($this) {
            self::Normal => app(EnvioNormal::class),
            self::Urgente => app(EnvioUrgente::class),
        };
    }
}
```

02 · SE VALIDA Y SE USA

```
// FormRequest: el tipo viene validado
'tipo' => ['required', Rule::enum(TipoEnvio::class)],

// Controlador o acción
$tipo = TipoEnvio::from($request->validated('tipo'));
$precio = $tipo->estrategia()->precio();

// 'urgente' => 12000
// 'normal'  => 5000
// 'express'  => 422: no es un caso del enum
```

EJEMPLO Y RESULTADO

El cliente elige urgente: TipoEnvio::Urgente selecciona EnvioUrgente y precio() devuelve 12.000. Si elige normal, 5.000.

TEN EN CUENTA

app() pide la clase al contenedor de Laravel: si su constructor necesita dependencias, se inyectan solas; con new habría que armarlas a mano.

EVENTO + LISTENER

La acción avisa con un evento; un listener reacciona

ANTIPATRÓN: LA ACCIÓN HACE TODO

```
class CreateArticle
{
    public function handle(array $data): Article
    {
        $article = Article::create($data);
        Mail::to('revisor@example.com')->send(
            new ArticleCreatedMail($article)
        );
        Log::info('Artículo creado');
        Cache::forget('articles');
        return $article;
    }
}
// cada reacción nueva obliga a editar la acción
```

PATRÓN: EVENTO Y LISTENER

```
// la acción solo avisa que ocurrió
event(new ArticleCreated($article));

// app/Listeners/NotifyReviewer.php
class NotifyReviewer
{
    public function handle(ArticleCreated $e): void
    {
        Mail::to('revisor@example.com')->send(
            new ArticleCreatedMail($e->article)
        );
    }
}
// Laravel lo descubre por el tipo de handle()
```

EJEMPLO Y RESULTADO

ArticleCreated solo transporta el artículo. Correo, log y caché son listeners aparte; la acción no sabe quién reacciona.

TEN EN CUENTA

Úsalo cuando algo ocurrió y otras partes pueden reaccionar. Si un paso debe ocurrir sí o sí dentro de la transacción, llámalo directamente.

RESUMEN

Cuándo usar cada práctica

Práctica	Úsala cuando	Evítala cuando
Permisos y policy	Se decide qué puede hacer cada usuario	Solo cambia la pantalla según el rol
FormRequest	Toda ruta que recibe datos	Reglas que consultan mucho: van a la acción
Resource	Un modelo sale por la API	Solo se devuelve un mensaje
Builder	La condición se repite o tiene nombre	Consulta de una sola vez
Enum	Pocos valores cerrados	El usuario crea los valores: tabla
Acción y DTO	Varias escrituras o una transacción	CRUD sencillo de una tabla
Strategy	Variantes que van creciendo	Dos variantes que no cambian
Evento y listener	Otras partes pueden reaccionar	Pasos obligatorios de la transacción

Ninguna práctica se usa por moda: cada una resuelve un problema concreto.

CIERRE

Antes de fusionar un cambio

- ¿La ruta autoriza en el servidor y hay una prueba de 401, 403 y del caso permitido?
- ¿La entrada pasa por un FormRequest y la salida por un Resource?
- ¿Los valores cerrados son enums, y las consultas repetidas, métodos de un builder?
- ¿El controlador solo orquesta, y la operación compuesta vive en una acción?
- ¿Las migraciones nuevas son repetibles y ninguna edita una ya ejecutada?